38. Databases: SQL Drivers, Transactions, Migrations

38. Databases: SQL Drivers, Transactions, Migrations

Phase: Phase 6 — Systems and infrastructure
Estimated time: 2 days
Slug: `38-databases-sql`

Learning objectives

- Use database/sql with a driver
- Wrap multi-step writes in transactions
- Think in migrations for schema change
- Avoid SQL injection with parameters

Prerequisites

Complete prior numbered lessons. Type every example yourself — reading Go is not writing Go.

Why this matters

database/sql is a toolkit, not an ORM. Correct parameters, transactions, and connection pools keep data safe under concurrency.

This lesson is a building block for later modules in the Goodway path. If you only skim, the exercises will feel like busywork; if you type the examples and narrate them aloud, the same patterns will reappear in HTTP handlers, concurrency, and AI service glue.

Core concepts

- Use placeholders `?`/`$1` — never string-build SQL.
- Query/QueryRow/Exec and always check errors.
- Tx: Begin → work → Commit/Rollback.
- sql.DB is a pool; SetMaxOpenConns thoughtfully.
- Scan into variables carefully; NULL needs sql.Null*.

Hold these in working memory while you code. Each bullet should map to something you can demonstrate with a three-line program or a failing compile error.

Worked Go examples

Create a scratch module and type this yourself (do not paste blindly):

```
mkdir -p /tmp/gw-databases: && cd /tmp/gw-databases:
go mod init example.com/scratch
```

Then create `main.go`:

```
package main
```

```
import "fmt"

// Conceptual parameterized query shape:
func FindName(id int) string {
    // db.QueryRow("SELECT name FROM users WHERE id=$1", id).Scan(&name)
    _ = id
    return "Ada"
}

func main() { fmt.Println(FindName(1)) }
```

Run it:

```
go run .
```

Expected output:

```
Ada
```

If your output differs, diff character-by-character — especially whitespace and types. Then change one line deliberately and predict the new output before running.

Mini-lab (`go run` steps)

1. Write a function that takes query args separately from SQL.
2. Sketch transaction rollback on error.
3. List what happens if you ignore Rows.Err().

Write down one sentence explaining *why* the output matches the code before you move on. Time-box the lab to 15–25 minutes; struggle is part of encoding the skill.

Curriculum notes (expanded from the compact draft)

database/sql

```
db, err := sql.Open("pgx", dsn)
db.SetMaxOpenConns(20)
row := db.QueryRowContext(ctx, `SELECT name FROM users WHERE id=$1`, id)
```

Always use contexts and placeholders — never string-concatenate user input into SQL.

Transactions

```
tx, err := db.BeginTx(ctx, nil)
defer tx.Rollback()
// execs...
return tx.Commit()
```

Migrations

Treat schema as versioned code (golang-migrate, goose, atlas). Expand/contract for zero-downtime.

Failure modes to watch while coding

- sql.Open without Ping
- Holding transactions open during HTTP calls to others
- SELECT * forever

When a test fails, ask which failure mode you hit before changing code at random.

Idioms vs Python pitfalls

- DB-API placeholders ≈ Go's — still don't f-string SQL.
- ORMs hide pools; learn sql.DB semantics anyway.
- context.Context on QueryContext for timeouts.

When you catch yourself reaching for a Python habit, pause and rewrite the Go way once on purpose — even if both would "work". The goal is a Go-shaped intuition.

Going deeper

Migrations belong in versioned files; apps should not ad-hoc ALTER in request paths.

Practice checklist

Before the exercise, confirm you can:

1. Explain "Databases: SQL Drivers, Transactions, Migrations" aloud in under two minutes without notes.
2. Re-type the worked example from memory and match the expected output.
3. Name the Go standard library packages touched by this lesson.
4. List one production bug this lesson's rules prevent.
5. Contrast the Go approach with the Python instinct listed above.
6. Write a three-line cheat sheet: one idiom, one pitfall, one command to remember.

If any item fails, re-run the mini-lab once more, then attempt the exercise tests.

Transfer to production

Ask where this shows up outside toys: config loading, API handlers, data pipelines, CI scripts, or on-call debugging. Write one concrete two-sentence scenario tying the worked example to that scenario. Tomorrow, when you rewrite the exercise from memory, check whether the code still fits that framing.

Common mistakes (especially from Python instincts)

- sql.Open without Ping
- Holding transactions open during HTTP calls to others
- SELECT * forever

Exercises

Work in `exercises/38-databases-sql/`. Read that folder's `README.md` before you code.

What you will build

Implement parameterized helpers / scans as the exercise defines.

Study loop

- **A (Drill):** make unit tests pass with the simplest correct code.
- **B (Apply):** extend with an edge case called out in the exercise README.
- **C (Challenge):** rewrite your solution from memory the next day without looking.

Do not open `solutions/38-databases-sql/` until you have an honest failing attempt.

Self-check

Answer without scrolling up:

1. Why placeholders?
2. What does Rollback on defer do after Commit?
3. Expand/contract meaning?

Further reading

- https://pkg.go.dev/database/sql